Applied Machine Learning for Business Analytics

Lecture 8: Agent: Tool

Lecturer: Zhao Rui

The most viral Agent

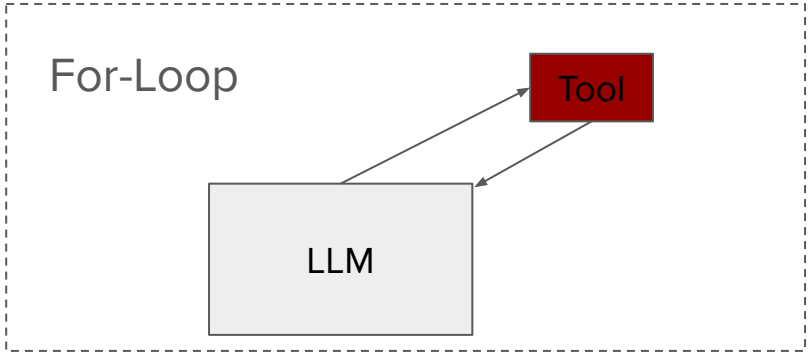
openclaw

Your personal, open source AI assistant.

Sponsor

15k followers | https://openclaw.ai | @openclaw | https://openclaw.ai | https://molty.me | peter@openclaw.ai

The most simple Agent



Agenda

1. Tools
2. MCP

1. Tools

Tools

- In nature, LLM is only able to do next token prediction
  - A brilliant brain with no hands
- By enabling external tools, agents can have more capabilities.
- Tools can be classified into two types:
  - Reading
    - SQL executor, API call & web crawl-> have more accurate and up-to-date info
    - Calculator, unit converters, code interpreters, unit converters -> reduce hallucination
  - Writing
    - Modify data sources, email API sending responses, modify code etc -> enable automation in workflow

Tool calling is how we give the LLM hands — letting it trigger real actions in the real world, while the LLM stays in charge of *deciding what to do*.

Tool Calling

- Tools: functions made available to LLMs
- How to make it available to LLM: Tool Schema
  - An appropriate name
  - Relevant Parameters
  - Description of the tool's purpose
- Tool Calling
  - LLMs **do not execute the function**, it only generates a structured schema for the tool specifying
    - Tool name with the necessary parameters -> basic info to execute the tools

How the LLM knows when & how to use a tool

A tool definition looks like this

```
name: "get_weather"  ← identifier
 description: "Get current weather for a city"
← WHEN to call it
 parameters: {
  location: string
  ← WHAT to pass
}
```

Description = instruction to the model

The LLM reads the description to decide whether to call a tool. Write clear, specific descriptions — they are instructions for the model, not docs for humans.

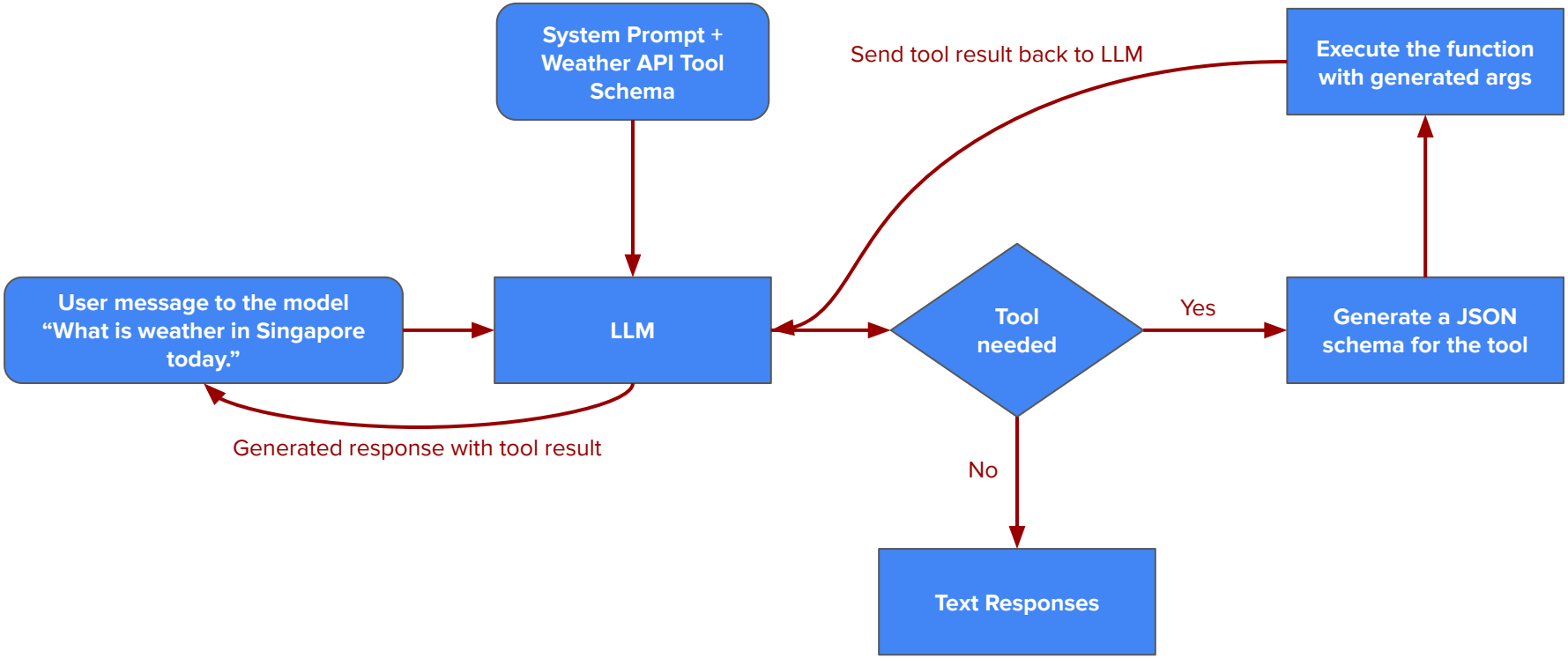
Parameters are filled from the conversation

location = "Singapore" is inferred automatically from the user's message and the parameter type.

No match → plain text, no forced call

If no tool description fits the question, the LLM answers normally in plain text.

Tool Calling



5-step loop - tool calling

Step 1

💬

User asks

Natural language question or instruction

→

Step 2

🤔

LLM picks a tool & responds

Decides which tool fits and what parameters to pass

→

Step 3

⚙️

App runs it

Your application executes the tool — API call, DB query, etc.

→

Step 4

📬

Result returns

Tool output is fed back to the LLM as context

→

Step 5

✅

LLM answers

Uses the real data to compose a grounded, accurate response

The LLM never *runs* the tool itself — it only *decides* to use one. Your application does the actual work and feeds the result back.

Example: what is the weather in Singapore

Step 1

User asks

What is the weather in Singapore?

→

Step 2

LLM picks a tool and responds

get_weather
location: Singapore

→

Step 3

App runs it

Hits weather API with location-Singapore

→

Step 4

Result returns

partly cloudy

→

Step 5

LLM answers

It is partly cloudy in Singapore right now

```
// What the LLM actually returns in Step 2:
{ "tool": "get_weather", "arguments": { "location": "Singapore" } }
```

How LLM have the Tool Calling Capability

Baked into the model via post-training

Modern LLMs (GPT-4, Claude, Gemini...) are specifically trained to:

✓ Read tool schemas — name, description, parameters

✓ Decide when a question needs a tool vs. a plain text answer

✓ Respond with structured tool calls (name + arguments) instead of string (usually json format)

Not a plugin or add-on — a core part of how the model works.

Activated per request — not globally

There is no "tool mode" switch. You activate it for a specific request by including tool definitions in that API call.

✓ Include tool definitions → LLM can return tool calls

✓ Omit them → LLM answers in plain text only

✓ Each request independently controls which tools are available

No extra config — just include the list in your API call.

Enabling tool calling = **a model trained for it** + **tool definitions included in the prompt**.

Training Samples for Tool Calling

```
Standard CHAT
Messages = [
{ "role": "user",
  "content": "help with order #123”},
{ "role": "assistant",
  "content": "I would be happy to help”},
]
```

```
Tool-calling CHAT
Messages = [
{ "role": "user",
  "content": "Status of order #123”},
{ "role": "assistant",
  "tool_call”: {
   “name”: “get_status”,
   “arguments”: ‘{“txn”:”T1001”}’}},
{ "role": "tool",
  "content": ‘{“status”:”paid”}’},
{ "role": "assistant",
  "content": "Transaction #123: Paid”},
]
```

Training samples must demonstrate **the complete cycle**. The model learns to: decide which tool to call, format the call correctly, then interpret the result.

Tutorial: https://huggingface.co/learn/agents-course/en/bonus-unit1/introduction

Prompt Template

Identity

You are a helpful assistant. You should think step by step.
Use tools only when they are necessary for the task. If a query can be answered directly, respond with a simple message instead of using tools. You have the following tools available to you.
Tools: [...]
A few shots: [(query;tool_calls), ...]

Instructions

Langchain’s Tool Calling: Big Picture

- In LangChain, we do not need to write the system prompt for tool calling manually. It is constructed automatically from two things we provide

Tool Definitions

Python functions decorated with @tool - their names, docstrings, and type hints.

bind_tools()

Seralizes those definitions into JSON Schema and injects them into every API call

Define Tools via Decorator

The @tool decorator reads three parts of the function and converts them into a JSON Schema the model can understand

```python
@tool
def get_current_weather(location: str, unit: str = "celsius") -> str:
    """Get current weather in a location."""
    ...

@tool
def calculate(expression: str) -> str:
    """Evaluate a math expression."""
    ...
```

Auto-generated JSON Schema

```
{
  "name": "get_current_weather",
  "description": "Get current
    weather in a location.",
  "parameters": {
    "location": {
      "type": "string"
    },
    "unit": {
      "type": "string",
      "default": "celsius"
    }
  }
}
```

LLMs uses func. Name to call the right tool

Description of what the tool does

Validate LLM' argument output

LLM can omit optional params.

Docstrings are the documentation for LLM. The LLM never reads the source code - only the schema. Write docstring as if briefing the LLM, not just colleagues

Create Hidden System Prompt: bind_tools()

```
model_with_tools = model.bind_tools(TOOLS)
```

Auto-Inject

You have access to these tools:

Tool: get_current_weather
  Description: Get current weather in a location.
  Parameters: { location: string, unit: string (default celsius) }

Tool: calculate
  Description: Evaluate a math expression.
  Parameters: { expression: string }

When calling a tool, respond with JSON only:
{"tool_name": "...", "arguments": {...}}

Usually invisible

Bind_tools() serializes all tool schemas and injects them into the LLM API call as special tools parameters (OpenAI) or an equivalent system-level prompt for other LLMs

OpenAI and Antropic

OpenAI

```python
response = client.responses.create(
    model="gpt-5",
    instructions="Respond only with a horoscope generated by a tool.",
    tools=tools,
    input=input_list,
)
```

Auto-Inject tool info into prompt for LLM call

Antropic

```python
import anthropic

client = anthropic.Anthropic()

response = client.messages.create(
    model="claude-opus-4-6",
    max_tokens=1024,
    tools=[
        {
            "name": "get_weather",
            "description": "Get the current weather in a given location",
            "input_schema": {
                "type": "object",
                "properties": {
                    "location": {
                        "type": "string",
                        "description": "The city and state, e.g. San Francisco, CA",
                    }
                },
                "required": ["location"],
            },
        }
    ],
    messages=[{"role": "user", "content": "What's the weather like in San Francisco?"}],
)
print(response)
```

The Full Prompt

```
messages = [HumanMessage(content=question)]
```

Auto-injected Tool Schema Block

```
You have access to these tools:

Tool: get_current_weather
  Description: Get current weather in a location.
  Parameters: { location: string, unit: string (default celsius) }

Tool: calculate
  Description: Evaluate a math expression.
  Parameters: { expression: string }

When calling a tool, respond with JSON only:
{"tool_name": "...", "arguments": {...}}
```

HumanMessage

```
What is the weather like in Singapore?
```

Optional Custom System Message

- Customizing system prompt
  - Role definition - who is the assistant?
  - Goal and constraints - when should it use tools?
  - Few-shot examples - show example tool calls?

```
from langchain_core.messages import SystemMessage

messages = [
    SystemMessage(content="""You are a helpful assistant.
Always use the available tools when the question requires real data.
If the user asks about weather, use get_current_weather.
If the user asks for a calculation, use calculate."""),
    HumanMessage(content=question)
]
```

You have access to these tools:

Tool: get_current_weather
  Description: Get current weather in a location.
  Parameters: { location: string, unit: string (default celsius) }

Tool: calculate
  Description: Evaluate a math expression.
  Parameters: { expression: string }

When calling a tool, respond with JSON only:
{"tool_name": "...", "arguments": {...}}

You are a helpful assistant.
Always use the available tools when the question requires real data.
If users ask about weather, use get_current weather,
If users ask for a calculation, use calculate

What is the weather like in Singapore?

Tool Callings Loop

Step 2

Send query + Tools

HumanMessage + Tool Schemas sent to model

```
messages = [HumanMessage(content=question)]

for _ in range(max_rounds):
    response = model_with_tools.invoke(messages)
```

Step 3

Model decides

Does response.tool_calls have entries?

- no->final answer
- yes -> continue

```
response = model_with_tools.invoke(messages)
print("Model:", response.content)

# STEP 3: If no tool calls, we have a final answer
if not response.tool_calls:
    print("Answer:", response.content or "(no content)")
    return

messages.append(response)
```

Step 4

Execute tool

The Python code calls run_tool(func_name, arg) - LLM is NOT involved here

```
# STEP 4: Execute each tool call in application code
for tool_call in response.tool_calls:
    name = tool_call["name"]
    args = tool_call.get("args", {})
    print(f"Tool requested -> {name}({args})")
    tool_result = run_tool(name, args)
```

Step 5

Return Result

Toolmessage appended to conversion. LLM reads it and gives a final answer.

```
# STEP 5: Send tool result back to the LLM via ToolMessage
messages.append(
    ToolMessage(content=tool_result, tool_call_id=tool_call["id"])
)
```

Message History as the loop runs

```
# Initial
messages = [HumanMessage("What's the weather in London?")]

# After model responds with tool_calls
messages += [AIMessage(tool_calls=[{"name": "get_current_weather", "args": {"location": "London"}}])]

# After your code executes the tool
messages += [ToolMessage("London: 15.0C, sunny", tool_call_id="...")]

# Model now has context to give final answer
```

Why the Docstring Matters

```python
@tool
def get_current_weather(location: str, unit: str = "celsius") -> str:
    """Get current weather in a location."""
    ...

@tool
def calculate(expression: str) -> str:
    """Evaluate a math expression."""
    ...
```

This is ALL the model knows about this tool

Bad	Good
“”” Does weather stuff.”””	“”” Get current weather in a location. Returns temperature and conditions”””
“”” Math.”””	“”” Evaluate a math expression like ‘4*7/3’. Returns the numeric result.”””

Tool Callings: Yes

```
result = model_with_tools.invoke("What is the weather like in Singapore")
print(result)
```

Generate a JSON schema for the tool

content=''
additional_kwargs={}
response_metadata={'*model*': 'qwen2.5:3b', '*created_at*': '2026-03-07T12:08:30.513783Z', '*done*': True, '*done_reason*': 'stop', '*total_duration*': 654724833, '*load_duration*': 30845750, '*prompt_eval_count*': 215, '*prompt_eval_duration*': 184000000, '*eval_count*': 21, '*eval_duration*': 437000000, '*logprobs*': None, '*model_name*': 'qwen2.5:3b'}
id='run--019cc832-f9a1-7b31-beca-a5571dcea975-0'
tool_calls=[{'*name*': 'get_current_weather', '*args*': {'location': 'Singapore'}, '*id*': 'd759987e-f8e8-497f-8233-26aabc68e149', '*type*': 'tool_call'}]
usage_metadata={'*input_tokens*': 215, '*output_tokens*': 21, '*total_tokens*': 236}

```python
# STEP 4: Execute each tool call in application code
for tool_call in response.tool_calls:
    name = tool_call["name"]
    args = tool_call.get("args", {})
    print(f"Tool requested -> {name}({args})")
    tool_result = run_tool(name, args)

    # STEP 5: Send tool result back to the LLM via ToolMessage
    messages.append(
        ToolMessage(content=tool_result, tool_call_id=tool_call["id"])
    )
```

Tool Callings: No

```
result = model_with_tools.invoke("Hello, how are you?")
print(result)
```

```
# STEP 3: If no tool calls, we have a final answer
if not response.tool_calls:
    print("Answer:", response.content or "(no content)")
    return
```

content="I'm here to help with any questions or tasks you have. How can I assist you today?"
additional_kwargs={}
response_metadata={'model': 'qwen2.5:3b', 'created_at': '2026-03-07T12:08:31.634181Z', 'done': True, 'done_reason': 'stop', 'total_duration': 537898250, 'load_duration': 12316459, 'prompt_eval_count': 213, 'prompt_eval_duration': 93000000, 'eval_count': 21, 'eval_duration': 430000000, 'logprobs': None, 'model_name': 'qwen2.5:3b'}
id='run--019cc832-fe76-70c0-90c8-6fd348e8a280-0'
usage_metadata={'input_tokens': 213, 'output_tokens': 21, 'total_tokens': 234}

Common Misconceptions

Myth	Fact
I need to write a system prompt listing my tools	bind_tools() injects the tool description automatically
Docstrings are just for developers	LLM never read source code. Only the schema +doctstrings is sent.
LLM executes the tool	Your code executes tools
One round-trip is enough	User query->LLM tool request->tool results->LLM final answer

Best Practices for Tool Design

- Detailed Description
  - The description is the only way the model knows when and how to use a tool:
    - Be Verbose: write 3-4 sentences per description. Do not just say what it does; say when it should be used and when it should not
    - Explain Parameters: describe every param's meaning, expected format, and how it affects the outcome
    - Clarify Boundaries: if you have two similar tools (`search_email` and `search_files`), explicitly explain the difference in their description)
- Consolidate for "Selection Simplicity"
  - Avoid Tool Bloat: Having too many narrow tools (`get_user_name` and `get_user_email`and `get_user_id`) confuses the model
  - Group related Actions: create a single, more capable tool with an `action` parameter (a `manage_user` tool with an enum for `get_ifno`, `update` or `delete`). This reduces "selection ambiguity)

Best Practices for Tool Design

- Use Meaningful Namespacing
  - Prefixing: if you application connects to multiple services, use prefixes (`slack_send_message` and `github_create_issue`). This makes the tool's origin and purpose instantly recognizable
- Avoid Overloading the Model with Too Many Tools
  - Start with 3-5 well-scoped tools. Only add more when we are confident the model is handling current tools well
- Favour Models With Native Tool Calling Support
  - Not all language models are equally good at tool calling. OpenAI models with function calling or Antropic's Claude with tool use capabilities

More details: https://platform.claude.com/docs/en/agents-and-tools/tool-use/implement-tool-use

2. MCP

Model Context Protocol

ANTHROP\C

Research Economic Futures Commitments Learn News Try Claude

Announcements

Introducing the Model Context Protocol

25 Nov 2024

Today, we're open-sourcing the Model Context Protocol (MCP), a new standard for connecting AI assistants to the systems where data lives, including content repositories, business tools, and development environments. Its aim is to help frontier models produce better, more relevant responses.

As AI assistants gain mainstream adoption, the industry has invested heavily in model capabilities, achieving rapid advances in reasoning and quality. Yet even the most sophisticated models are constrained by their isolation from data—trapped behind information silos and legacy systems. Every new data source requires its own custom implementation, making truly connected systems difficult to scale.

MCP addresses this challenge. It provides a universal, open standard for connecting AI systems with data sources, replacing fragmented integrations

MCP is a protocol which will make LLM to use various kinds of tools better

What if tools weren’t hard-coded?

📱 Hard-coded tools (direct)

Your LLM apps were using a fixed set of tools written by your team. Adding a new tool or modifying a new tool means changing the code and redeploying. Every app that needs the same copy.

🧩 MCP: tools as plugins

Your LLM apps discovers tools from **external servers** at runtime — like installing apps on your phone. New tools = connect to a new server. No code change required.

MCP (Model Context Protocol) is an open standard that lets your LLM apps discover and run tools from external servers — without changing your application code each time a new tool is needed.

Think of it like: direct tool calling is a custom remote control you built for one specific TV. MCP is a **universal remote standard** — any device that follows the standard works with any compatible remote.

MCP: same loop, two things change

Step 1

🔍

Discover tools from MCP servers

LLM apps/MCP Hosts ask MCP servers: what tools do you have?

→

Step 2

🤔

LLM picks a tool & responds

Same as before

→

Step 3

⚙️

Route call to MCP server

LLMs forwards to the server that owns that tool; server runs it

→

Step 4

📬

Result returns

Tool output is fed back to the LLM as context

→

Step 5

✅

LLM answers

Uses the real data to compose a grounded, accurate response

MCP only change: tool discovery and tool execution. This protocol standardizes the interface of tools towards LLM: how tools are discovered, called, and how results are returned

Director tool calling vs MCP

Dimension	Direct Tool Calling	With MCP
Where tools live	Hard-coded in your application	Hosted by external MCP servers (local or remote)
Who maintains them	You, the app developer	Server operator — your team, another team, or third party
Discovery	Static — known at build time	Dynamic — discovered at runtime
Reusability	Coupled to your app only	Any MCP-compatible client can connect and reuse
Complexity	Simpler — one process, no extra infra	More moving parts — servers must be running and reachable
Best for	Prototypes, small apps, tightly-coupled logic	Shared capabilities, multi-team, third-party integrations

MCP in action: a fin analyst

🧠 **LLM**

Picks tools · generate answers

↕ 2. sends tool schemas · 5. returns final answer

📊 **Portfolio MCP Server**

get_holdings
get_pnl_summary
get_trade_history

1. list tools
↔
4. call tool

🔧 **Your App/MCP Host**

discovers tools
routes calls
passes results

1. list tools
↔
4. call tool

📈 **Market Data MCP Server**

get_quote
get_historical_prices
get_fx_rate

Final answer ↓ · ↑ user question

👤 **User**

"What's the current value of my AAPL position?"

MCP in action: a fin analyst

1. At the beginning: App discovers 6 tools from 2 MCP servers (3 portfolio tools + 3 market data tools)

2. App sends: user message "What's my AAPL position?" + all 6 tool schemas → LLM

3. LLM returns two tool calls: get_holdings(AAPL) and get_quote(AAPL)

4. App routes to the right server:
   📊 Portfolio → 150 shares @ avg $178.50
   📈 Market Data → $242.10, MTD +5.3%

✔ LLM generates: "You hold 150 shares of AAPL at an average cost of $178.50. Current price is $242.10 — position value ~$36,315, unrealized gain ~$9,540. Month-to-date AAPL is up 5.3%."

Your App wrote **zero lines** of portfolio or market data logic. It just discovered tools, routed calls, and passed results — the servers owned all the domain knowledge.

When to use direct tool vs MCP

Use Direct Tool Calling when...

- ✓ You're prototyping or building fast
- ✓ You have ≤ 5 tools and they're straightforward
- ✓ Tools are specific to this one app only
- ✓ You need lowest possible latency (no network hop)
- ✓ Tool logic is tightly coupled to your app's state

Use MCP when...

- ✓ Multiple apps or teams need the same tools
- ✓ Different teams own different capabilities
- ✓ You want to add tools without redeploying your app
- ✓ You're integrating with third-party providers
- ✓ You want a shared standard, not ad-hoc wiring

Practical rule: Start with direct. Graduate to MCP when you find yourself copying the same tool logic across multiple apps — or when team boundaries demand it.

Summary

- The 5-step loop is universal
  - ask->pick->run->return->answer
- LLM decides, application acts
  - The model never directly calls APIs or queries databases. It returns structured instructions that your code executes - then feeds the result back
- MCP is the standardisation layer
  - Tools live outside your app - discoverable, reusable, maintainable by difference teams
- Start simple; graduate when you need scale
  - Direct tool calling ships faster. MCP pays off when same capability needs to be shared across functions

Next Class: Agent: Thinking Patterns